Technisch ontwerp

Restaurantreserveringen, bestellingen en menukaart

Ali Khoshkam

Leerlijn Backend (30 EC) – Eindopdracht versie 4.3

Inleverdatum: 8 november 2026

Documentversie: 1.1 | LaTeX-bron 8 november 2026

Inhoudsopgave

1 Inleiding en user stories **2**

2 Architectuur **2**

3 Functionele eisen — klant **2**

4 Functionele eisen — personeel **3**

5 Niet-functionele eisen **3**

6 Klassendiagram van het domein **4**

7 Sequentie 1 — reservering maken **5**

8 Sequentie 2 — factuur maken **5**

9 Technische afspraken, verificatie en bronnen **5**

9.1 Bronnen 6

1 Inleiding en user stories

Ik bouw een API voor een restaurant. Reserveringen, de menukaart en de rekening stonden eerst los van elkaar. In deze applicatie horen ze bij één database. Een klant ziet alleen de eigen reserveringen. Personeel beheert de kaart, de tafels, de bestellingen en de facturen. Inloggen loopt via Keycloak. De site *The Table* zit in dezelfde JAR en is Engelstalig.

US1 Als klant wil ik mij registreren en veilig inloggen, zodat ik mijn eigen reserveringen kan beheren zonder toegang tot gegevens van andere klanten.

US2 Als klant wil ik een beschikbare tafel reserveren, mijn reservering wijzigen en annuleren, zodat mijn restaurantbezoek passend en actueel is.

US3 Als personeelslid wil ik bestellingen met meerdere producten beheren en daarvoor een factuur met betaalstatus maken, zodat ik de bediening en afrekening centraal kan afhandelen.

US4 Als personeelslid wil ik producten, categorieën, tafels en productafbeeldingen beheren, zodat klanten en medewerkers actuele restaurantgegevens gebruiken.

Facturen en tafels zijn ondersteunend aan reserveren en bestellen. Naast accounts bevat het systeem drie operationele gebieden: reserveringen, bestellingen/facturen en menukaartbeheer met upload en download.

2 Architectuur

De code staat in vier lagen:

- **Controller** vertaalt HTTP naar gevalideerde DTO-input en zet statuscodes. Entiteiten verlaten deze laag niet.
- **Service** beheert transacties, bedrijfsregels en mapping via `Views.of()`.
- **Repository** bevat Spring Data JPA, unieke queries en `PESSIMISTIC_WRITE`-locks.
- **Model** bevat acht JPA-entiteiten met foreign keys, unieke constraints en versienummers.

`open-in-view` staat uit. JWT-validatie controleert handtekening, issuer, audience `restaurant-api` en subject. H2 is de standaarddatabase; PostgreSQL is optioneel via omgevingsvariabelen.

3 Functionele eisen — klant

Eis	Story	Acceptatiecriterium
F01	US1	Nieuwe accounts registreren via Keycloak en krijgen automatisch CUSTOMER.
F02	US1	Keycloak authenticeert en geeft een ondertekend access token voor restaurant-api.
F03	US1	Alleen een Keycloak-beheerder kent STAFF toe; zelfregistratie geeft geen personeelsrechten.
F04	US1	Een klant kan geen reservering van een ander lezen, wijzigen of annuleren (HTTP 403).
F05	US2	Een ingelogde gebruiker maakt een reservering met naam, tafel, UTC-begintijd, gasten en opmerkingen.

Eis	Story	Acceptatiecriterium
F06	US2	Alleen een toekomstige begintijd is geldig; eindtijd is begintijd plus twee uur.
F07	US2	Niet-inzetbare tafels en groepen groter dan de capaciteit worden geweigerd.
F08	US2	Overlappende actieve reserveringen voor dezelfde tafel worden geweigerd; aansluitende tijdvakken mogen.
F09	US2	Eigenaar en personeel wijzigen een actieve reservering; geannuleerde reserveringen blijven vergrendeld.
F10	US2	DELETE annuleert met behoud van historie en geeft het tijdvak vrij.
F11	US2	GET toont klanten alleen eigen reserveringen en personeel de volledige lijst.

4 Functionele eisen — personeel

Eis	Story	Acceptatiecriterium
F12	US3	Personeel beheert een bestelling met tafel en 1 tot 100 productregels.
F13	US3	Een product mag per bestelling eenmaal voorkomen (hoeveelheid 1–100); onbeschikbare producten worden geweigerd.
F14	US3	De server berekent het totaal uit vastgelegde eenheidsprijzen maal aantallen (`BigDecimal`).
F15	US3	Bestaande bestelregels behouden productnaam en eenheidsprijs bij menukaartwijzigingen.
F16	US3	Maximaal één factuur per bestelling; het bedrag komt uit de bestelling.
F17	US3	Een gefactureerde bestelling kan niet worden gewijzigd of verwijderd zolang de factuur bestaat.
F18	US3	Personeel markeert een factuur als betaald; betaald kan niet terug of verwijderd.
F19	US3	Een onbetaalde factuur mag worden verwijderd, zodat de bestelling weer bewerkbaar is.
F20	US4	Personeel beheert categorieën en producten; verwijderen alleen indien ongebruikt.
F21	US4	Personeel beheert tafels; gebruikte tafels en gereserveerde capaciteit zijn beschermd.
F22	US4	Klanten en personeel lezen catalogus en tafels; mutaties vereisen STAFF.
F23	US4	Personeel uploadt per product één PNG/JPEG; klanten en personeel downloaden die.

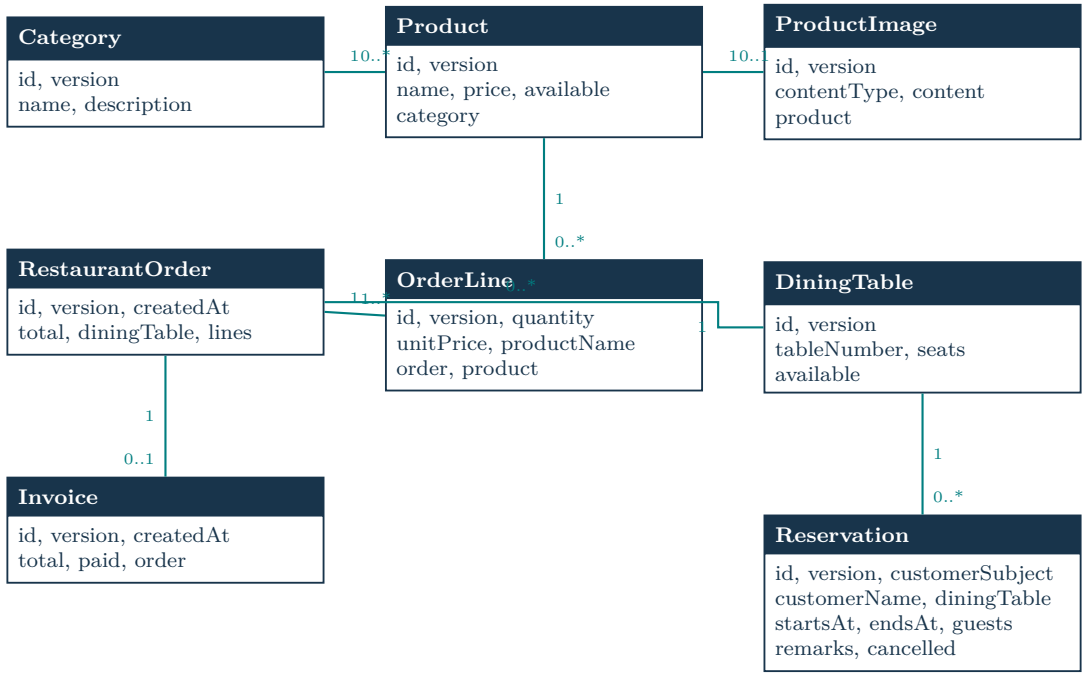
5 Niet-functionele eisen

Eis	Story	Acceptatiecriterium
N01	US1	JWT-handtekening, issuer, audience en verloopdatum worden gevalideerd; ontbrekend token geeft 401.

Eis	Story	Acceptatiecriterium
N02	US2	Reserveringscontrole en opslag gebeuren in één transactie met een schrijflock op de tafel.
N03	US3	Foreign keys, unieke factuur-orderkoppeling, versienummers en numerieke constraints bewaken integriteit.
N04	US4	Uploads maximaal 2 MiB en 16 miljoen pixels; paden komen nooit uit de bestandsnaam.
N05	US1–4	Java 21 / Spring Boot met gescheiden controllers, services, repositories en DTO-records.
N06	US1–4	Bekende fouten als 400/403/404/409/413/415; interne details blijven verborgen.
N07	US2–3	Maven verify: minstens twintig unit-tests en integratietests; PricingService en ReservationPolicy 100% line coverage.
N08	US1–4	JDBC is extern instelbaar; H2 standaard, PostgreSQL-driver meegeleverd.
N09	US4	`data.sql` laadt demonstratiegegevens zonder bestaande rijen te overschrijven.

6 Klassendiagram van het domein

Figuur 1 toont acht entiteiten met kardinaliteiten. Attributen sluiten aan bij de Java-klassen; methodes zijn weggelaten. `id` en `version` ondersteunen opslag en gelijktijdige wijzigingen.

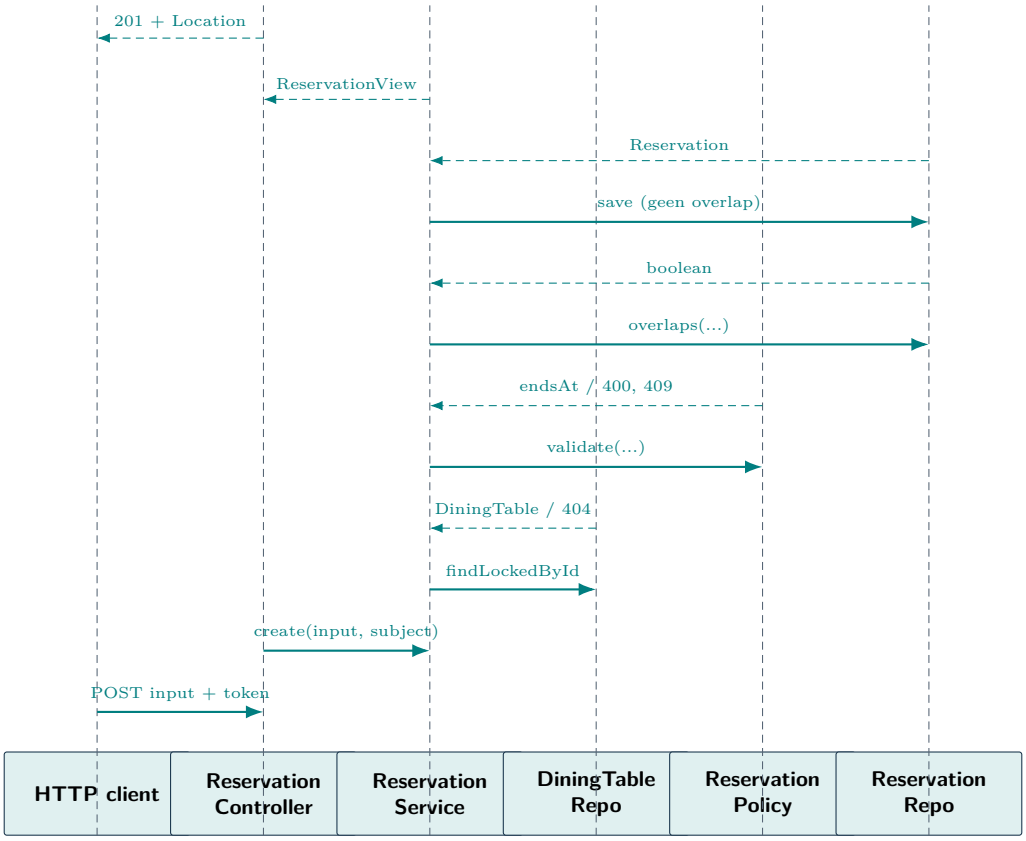


Figuur 1: Domeinmodel: één-op-één voor factuur en productafbeelding; overige relaties één-op-veel.

`OrderLine` koppelt bestelling en product met hoeveelheid en historische prijs. Een geannuleerde reservering blijft bestaan.

7 Sequentie 1 — reservering maken

Figuur 2 toont `POST /api/reservations`. Eigenaarschap komt uit het JWT-subject. De transactie houdt de tafellock vast tot opslag.



Figuur 2: Aanmaken van een reservering met tafellock en overlapcontrole.

Alternatieven: geen token 401; ontbrekende tafel 404; ongeldig tijdstip of capaciteit 400; tafel niet inzetbaar of overlap 409. Bij overlap wordt `save()` niet uitgevoerd. De integratietest bewijst één winnaar bij twee gelijktijdige boekingen.

8 Sequentie 2 — factuur maken

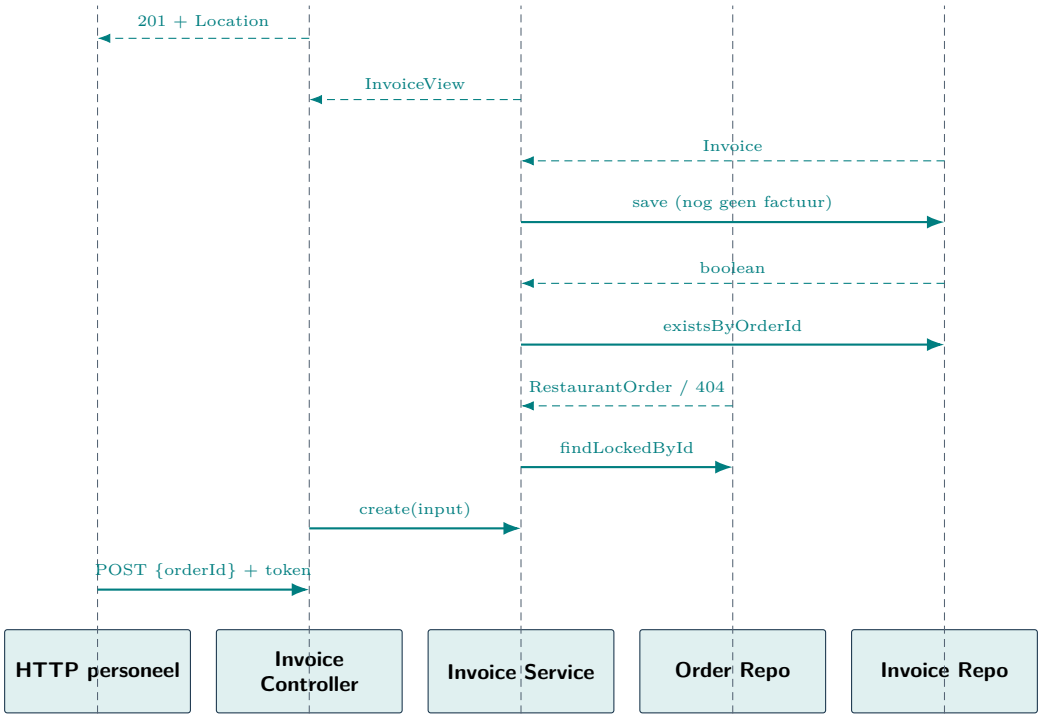
Figuur 3 toont `POST /api/invoices` (alleen STAFF). Het bedrag komt uit de bestelling, niet uit de requestbody.

Alternatieven: geen STAFF 403; bestelling ontbreekt 404; bestaande factuur 409. De unieke `order_id`-constraint is een tweede verdedigingslaag. `createdAt` komt uit `Clock`, `total` uit de bestelling, `paid` start als `false`.

9 Technische afspraken, verificatie en bronnen

Verwijderregels beschermen referenties: een categorie met producten, een gebruikt product en een gebruikte tafel leveren 409. Tijdwaarden zijn UTC zonder suffix. Bedragen zijn EUR-eindprijzen zonder btw-uitsplitsing.

Verificatie: `mvn clean verify` met Java 21 gaf 42 tests, 0 fouten (1 november 2026). JaCoCo bewaakt 100% line coverage voor `PricingService` en `ReservationPolicy`. Zie `docs/verification.md`.



Figuur 3: Aanmaken van een factuur met orderlock en unieke orderkoppeling.

9.1 Bronnen

1. Ali Khoshkam, *idee fase backend 2.pdf*, aangeleverd lokaal.
2. NOVI, *Eindopdracht Backend v4.3.pdf*, inclusief quickscan en rubric.
3. Spring Boot: `https://docs.spring.io/spring-boot/3.5/system-requirements.html`
4. Spring Security JWT: `https://docs.spring.io/spring-security/reference/6.5/servlet/oauth2/resource-server/jwt.html`
5. Keycloak-import: `https://www.keycloak.org/server/importExport`

Ik heb dit ontwerp zelf gemaakt. OpenAI Codex heeft me geholpen bij stukken tekst en uitleg. De prompts die ik zelf heb gestuurd staan in `docs/ai-prompts.md`. De diagrammen horen bij de klassen in de broncode.

Broncode: `https://github.com/Alikhoshkam1989/myRestaurant`